Bridging Python and SAP HANA
Inside the sqlalchemy-hana Open-Source dialect

Kai Harder, SAP

July 16, 2026

Agenda

01 Introduction into SQLAlchemy

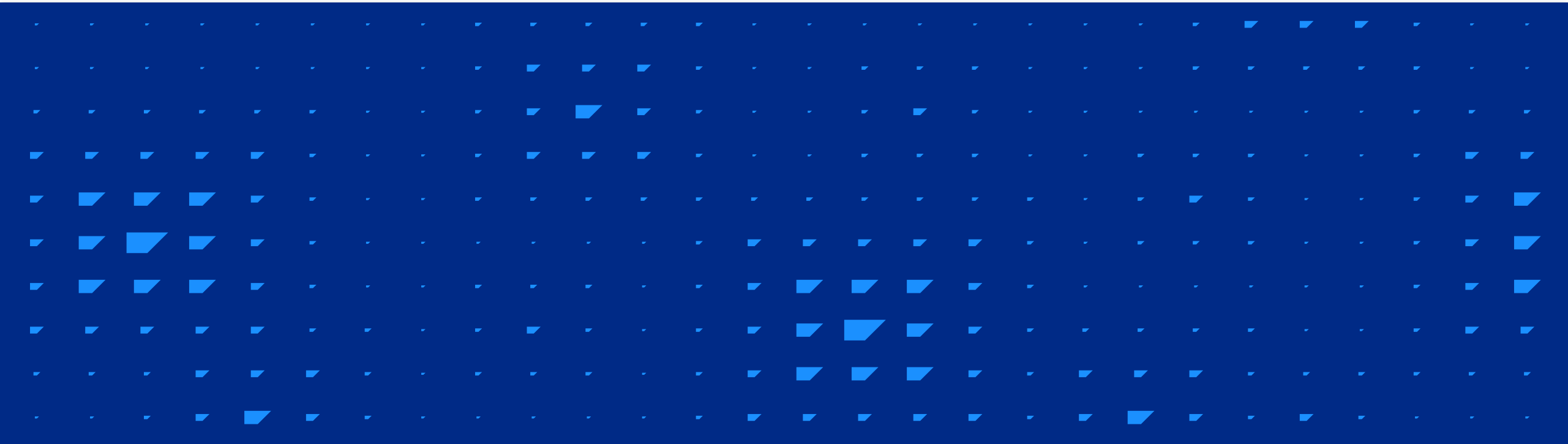
02 How does sqlalchemy-hana work

03 How to support HANA specifics

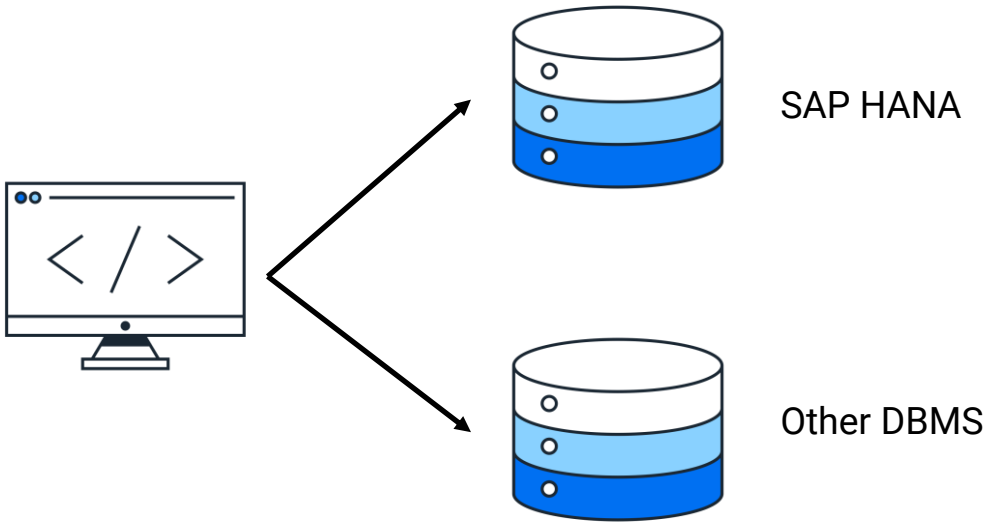
04 Schema migrations with alembic

05 New Features: async and vectors

Introduction into SQLAlchemy



The Problem



SQLAlchemy
ORM + write database-agnostic Python

Dialect
The translation layer between generic SQLAlchemy and DBMS specific SQL

Every database speaks a slightly different SQL dialect

- Pagination: `LIMIT/OFFSET` vs `ROWNUM` vs `FETCH FIRST`
- Booleans: native `BOOLEAN` vs `TINYINT` vs `NUMBER(1)`
- Identifiers: case-sensitive? Quoted? Max length?

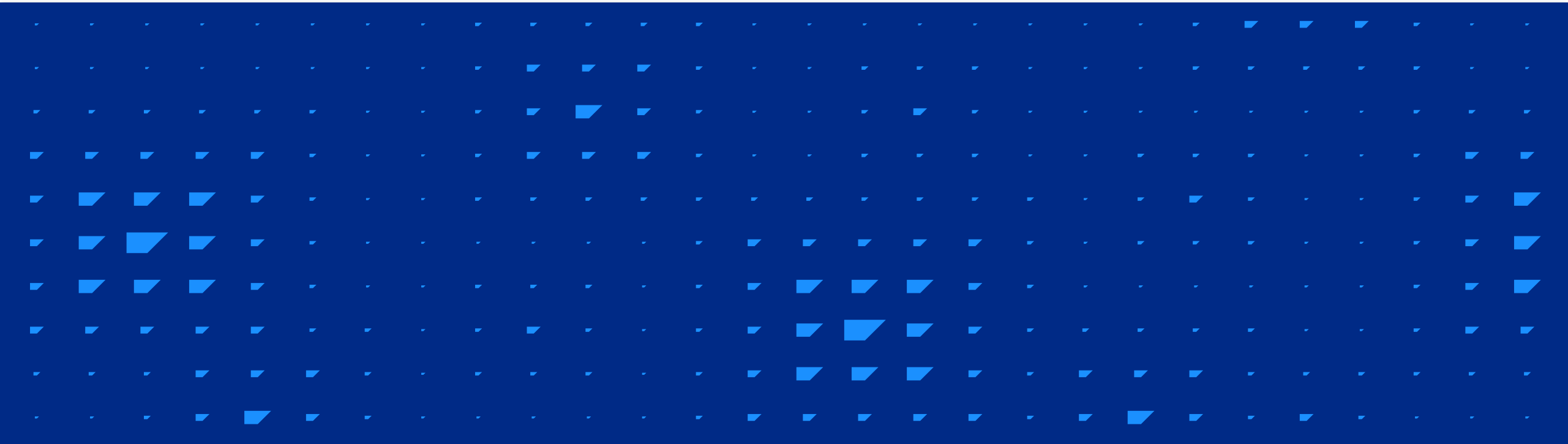
A Quick Example

```
class Employee(Base):
        __tablename__ = "employees"
        id = Column(Integer, primary_key=True)
        name = Column(String(100))
        department = Column(String(50))

session.add(Employee(name="Ada", department="Engineering"))
session.commit()

# SELECT * FROM employees WHERE department = 'Engineering'
engineers = session.scalars(
        select(Employee)
        .where(Employee.department == "Engineering")
).all()
```

How does sqlalchemy-hana work



Use sqlalchemy-hana

Install it using pip or uv: *pip install sqlalchemy-hana*

The user selects the dialect by the dburi:

```
engine = create_engine("hana://user:password@host:30015")
# or tenant DB
engine = create_engine("hana://user:password@host:port/DB")
# or HANA User Store key
engine = create_engine("hana:///?userkey=mykey")
```

The Dialect's Five Compiler Classes

HANAStatementCompiler

DML statements

HANATypeCompiler

Type name rendering

HANADDLCompiler

DDL statements

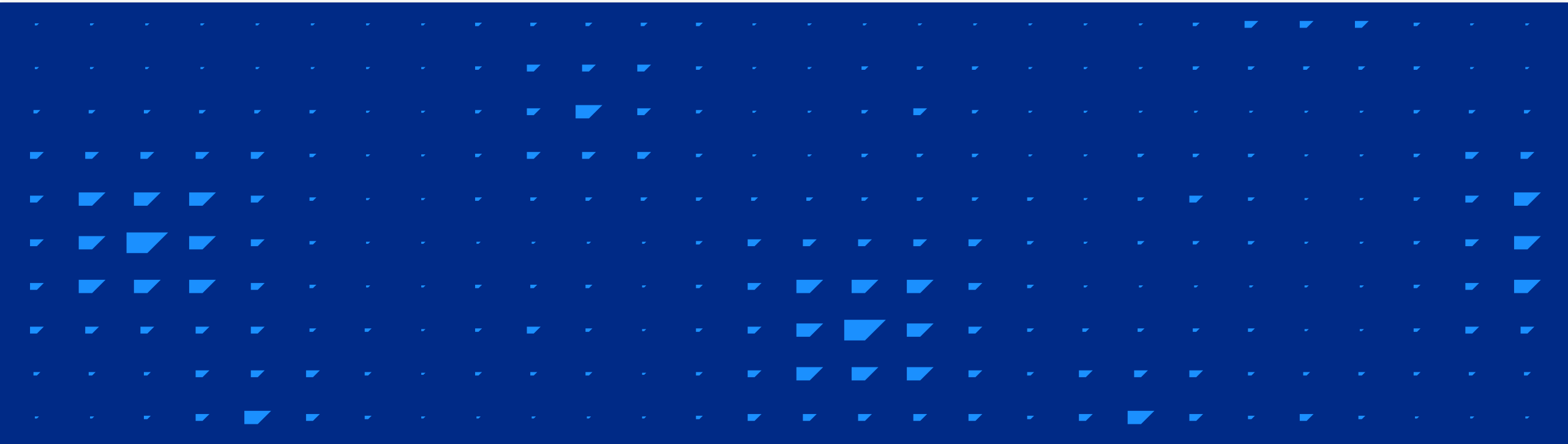
HANAIdentifierPreparer

Quoting identifiers

HANAExecutionContext

Sequences, etc.

How to support HANA specifics



SQL Generation

Every DBMS has its own way to write SQL, SAP HANA is no exception

- Every SELECT must have a FROM clause
- OFFSET without LIMIT is not possible

```
def default_from(self):
        return " FROM DUMMY"
```

Operators

IS DISTINCT FROM — not supported, must be rewritten

```
def visit_is_distinct_from_binary(self, binary, operator, **kw):
  left = self.process(binary.left)
  right = self.process(binary.right)
  return (
    f"(({left} <> {right} OR {left} IS NULL OR {right} IS NULL) "
    f"AND NOT ({left} IS NULL AND {right} IS NULL))"
  )
```

Identifiers

SQLAlchemy uses lowercase by default while HANA uses uppercase

```
# "mycolumn" in Python <-> "MYCOLUMN" in HANA's SYS.TABLE_COLUMNS
def normalize_name(self, name):
  if name.upper() == name and not self._requires_quotes(name.lower()):
    return name.lower() # MYCOLUMN -> mycolumn
  elif name.lower() == name:
    return quoted_name(name, quote=True) # preserve lowercase with quotes
  return name

def denormalize_name(self, name):
  if name.lower() == name and not self._requires_quotes(name.lower()):
    return name.upper() # mycolumn -> MYCOLUMN
  return name
```

Types

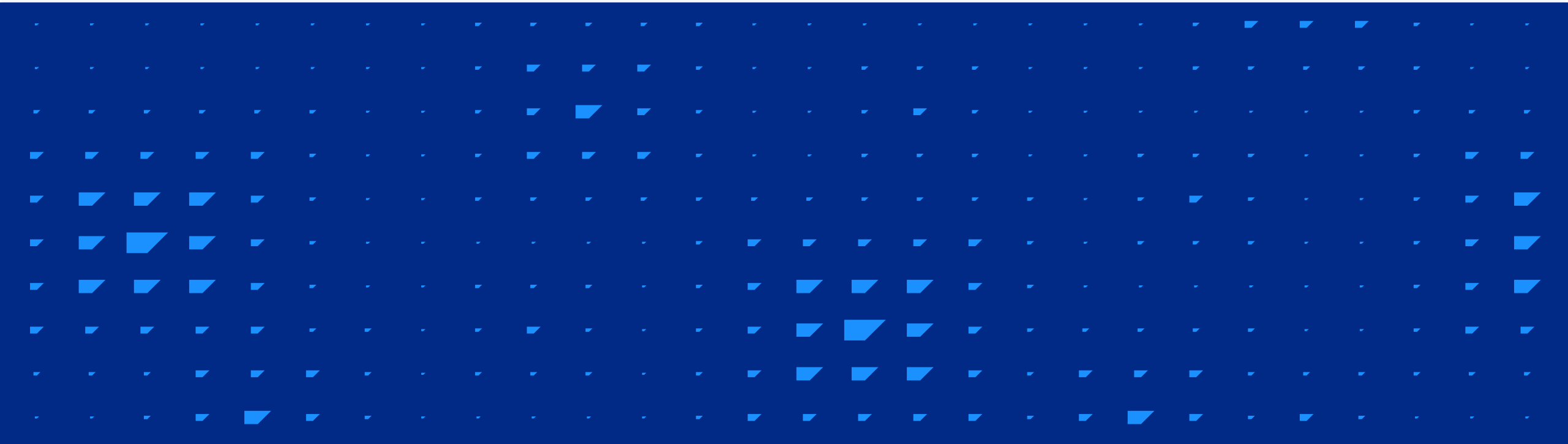
SQLAlchemy types are transformed into the SAP HANA equivalent

- String → NVARCHAR
- DateTime → TIMESTAMP

Some types are not supported by SQLAlchemy → own implementation

- TINYINT
- ALPHANUM
- SMALL_VECTOR

Schema migrations with alembic



Schema Migrations

Version-controlled schema changes - like git for your database

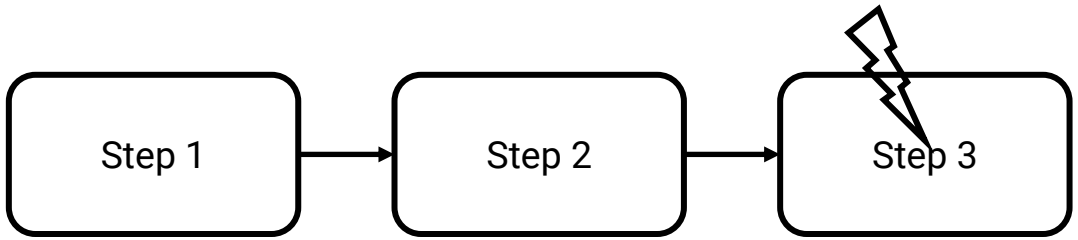
```
def upgrade():
  op.add_column("users", Column("email", String(200)))
def downgrade():
  op.drop_column("users", "email")
```

Alembic calls sqlalchemy-hana for concrete compilations

```
@compiles(AddColumn, "hana")
def visit_add_column(element, compiler, **kw) -> str:
  table = alter_table(compiler, element.table_name, element.schema)
  column = compiler.get_column_specification(element.column, **kw)
  return f"{table} ADD ({column})"
```

Transactional DDL

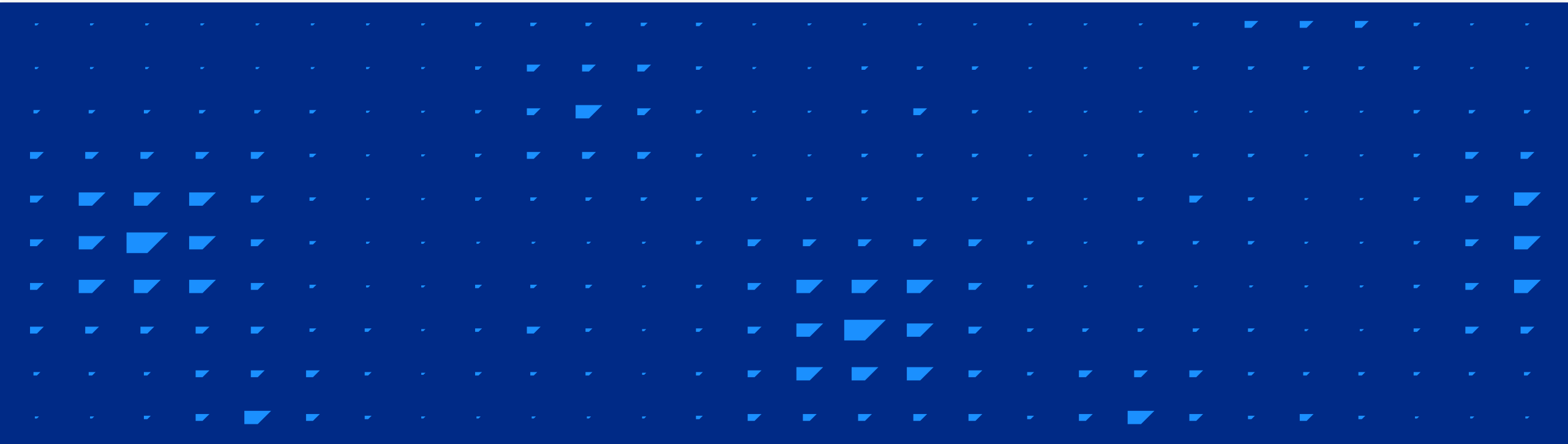
Scenario: multiple DDL statements in one migration



Most databases auto-commit DDL

With transactional DDL HANA can wrap it in a transaction

New Features: async and vectors



Async Support

Why async? FastAPI, asyncio apps, not blocking the event loop on slow queries.

```
from sqlalchemy.ext.asyncio import create_async_engine

engine = create_async_engine("hana+aiohdbcli://user:pass@host:30015")
async with AsyncSession(engine) as session:
  result = await session.execute(select(User))
  users = result.scalars().all()
```

Vector Support

SAP HANA has a native vector type → fully suppored by sqlalchemy-hana

```
# Get vectors as Python lists (default)
engine = create_engine("hana://...", vector_output_type="list")
# Get as tuples (immutable)
engine = create_engine("hana://...", vector_output_type="tuple")
# Get as memoryview (zero-copy, good for numpy)
engine = create_engine("hana://...", vector_output_type="memoryview")
```

Use Vectors

```
from sqlalchemy_hana.types import REAL_VECTOR
from sqlalchemy_hana.functions import cosine_similarity, l2distance

class Document(Base):
   __tablename__ = "documents"
   id = Column(Integer, primary_key=True)
   text = Column(NCLOB)
   embedding = Column(REAL_VECTOR[list[float]](1536))

# Find 5 documents most similar to a query embedding
stmt = (
   select(Document.id, Document.text)
   .order_by(cosine_similarity(Document.embedding, query_vector).desc())
   .limit(5) )
results = session.execute(stmt).all()
```

Resources

Important Links

SQLAlchemy	https://www.sqlalchemy.org/
sqlalchemy-hana	https://github.com/SAP/sqlalchemy-hana
hdbcli	https://help.sap.com/docs/SAP_HANA_CLIENT

Thank you.

Kai Harder

Development Expert
SAP

kai.harder@sap.com
https://github.com/kasium